



Cyberscope

A *TAC Security* Company

Audit Report

ViFoxCoin

December 2025

Network BSC

Address 0xec69f1351b66902cd5e79f0924a5ad049f682540

Audited by © cyberscope

Analysis

● Critical ● Medium ● Minor / Informative ● Pass

Severity	Code	Description	Status
●	ST	Stops Transactions	Acknowledged
●	OTUT	Transfers User's Tokens	Passed
●	ELFM	Exceeds Fees Limit	Passed
●	MT	Mints Tokens	Acknowledged
●	BT	Burns Tokens	Passed
●	BC	Blacklists Addresses	Passed

Diagnostics

● Critical ● Medium ● Minor / Informative

Severity	Code	Description	Status
●	ISU	Incorrect Stake Update	Acknowledged
●	AUIU	Abstract User ID Usage	Acknowledged
●	PRO	Possible Registration Overwrite	Acknowledged
●	TSI	Tokens Sufficiency Insurance	Acknowledged
●	CBSU	Calculations Before Supply Update	Acknowledged
●	CO	Code Optimization	Acknowledged
●	CCR	Contract Centralization Risk	Acknowledged
●	HV	Hardcoded Values	Acknowledged
●	MEE	Missing Events Emission	Acknowledged
●	PAMAR	Pair Address Max Amount Restriction	Acknowledged
●	PSU	Potential Subtraction Underflow	Acknowledged
●	RSK	Redundant Storage Keyword	Acknowledged
●	RVF	Redundant View Functionality	Acknowledged
●	UPS	Unsanitized Parameter Setters	Acknowledged

●	L02	State Variables could be Declared Constant	Acknowledged
●	L09	Dead Code Elimination	Acknowledged
●	L13	Divide before Multiply Operation	Acknowledged
●	L18	Multiple Pragma Directives	Acknowledged
●	L19	Stable Compiler Version	Acknowledged

Table of Contents

Analysis	1
Diagnostics	2
Table of Contents	4
Risk Classification	7
Review	8
Audit Updates	8
Source Files	8
Findings Breakdown	9
ST - Stops Transactions	10
Description	10
Recommendation	10
Team Update	11
MT - Mints Tokens	12
Description	12
Recommendation	13
Team Update	13
ISU - Incorrect Stake Update	14
Description	14
Recommendation	14
Team Update	15
AUIU - Abstract User ID Usage	16
Description	16
Recommendation	17
Team Update	17
PRO - Possible Registration Overwrite	18
Description	18
Recommendation	19
Team Update	19
TSI - Tokens Sufficiency Insurance	20
Description	20
Recommendation	20
Team Update	21
CBSU - Calculations Before Supply Update	22
Description	22
Recommendation	23
Team Update	23
CO - Code Optimization	24
Description	24
Recommendation	24

Team Update	24
CCR - Contract Centralization Risk	25
Description	25
Recommendation	25
Team Update	25
HV - Hardcoded Values	26
Description	26
Recommendation	27
Team Update	27
MEE - Missing Events Emission	28
Description	28
Recommendation	28
Team Update	29
PAMAR - Pair Address Max Amount Restriction	30
Description	30
Recommendation	31
Team Update	31
PSU - Potential Subtraction Underflow	32
Description	32
Recommendation	32
Team Update	32
RSK - Redundant Storage Keyword	33
Description	33
Recommendation	34
Team Update	34
RVF - Redundant View Functionality	35
Description	35
Recommendation	35
Team Update	35
UPS - Unsanitized Parameter Setters	36
Description	36
Recommendation	36
Team Update	36
L02 - State Variables could be Declared Constant	37
Description	37
Recommendation	37
Team Update	37
L09 - Dead Code Elimination	38
Description	38
Recommendation	39
Team Update	39
L13 - Divide before Multiply Operation	40

Description	40
Recommendation	40
Team Update	40
L18 - Multiple Pragma Directives	41
Description	41
Recommendation	41
Team Update	41
L19 - Stable Compiler Version	42
Description	42
Recommendation	42
Team Update	43
Functions Analysis	44
Inheritance Graph	46
Flow Graph	47
Summary	48
Disclaimer	49
About Cyberscope	50

Risk Classification

The criticality of findings in Cyberscope's smart contract audits is determined by evaluating multiple variables. The two primary variables are:

1. **Likelihood of Exploitation:** This considers how easily an attack can be executed, including the economic feasibility for an attacker.
2. **Impact of Exploitation:** This assesses the potential consequences of an attack, particularly in terms of the loss of funds or disruption to the contract's functionality.

Based on these variables, findings are categorized into the following severity levels:

1. **Critical:** Indicates a vulnerability that is both highly likely to be exploited and can result in significant fund loss or severe disruption. Immediate action is required to address these issues.
2. **Medium:** Refers to vulnerabilities that are either less likely to be exploited or would have a moderate impact if exploited. These issues should be addressed in due course to ensure overall contract security.
3. **Minor:** Involves vulnerabilities that are unlikely to be exploited and would have a minor impact. These findings should still be considered for resolution to maintain best practices in security.
4. **Informative:** Points out potential improvements or informational notes that do not pose an immediate risk. Addressing these can enhance the overall quality and robustness of the contract.

Severity	Likelihood / Impact of Exploitation
● Critical	Highly Likely / High Impact
● Medium	Less Likely / High Impact or Highly Likely/ Lower Impact
● Minor / Informative	Unlikely / Low to no Impact

Review

Contract Name	ViFoxCoin
Compiler Version	v0.8.30+commit.73712a01
Optimization	200 runs
Explorer	https://bscscan.com/address/0xec69f1351b66902cd5e79f0924a5ad049f682540
Address	0xec69f1351b66902cd5e79f0924a5ad049f682540
Network	BSC
Symbol	VFX
Decimals	3
Total Supply	9.913.423,8

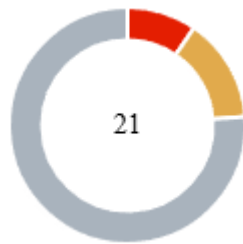
Audit Updates

Initial Audit	05 Dec 2025
---------------	-------------

Source Files

Filename	SHA256
ViFoxCoin.sol	1cea92eefdb34d9e4c8f3e60e862814dca29215e6c4f0d3ae675036610ef36cb

Findings Breakdown



● Critical	2
● Medium	3
● Minor / Informative	16

Severity	Unresolved	Acknowledged	Resolved	Other
● Critical	0	2	0	0
● Medium	0	3	0	0
● Minor / Informative	0	16	0	0

ST - Stops Transactions

Criticality	Critical
Location	ViFoxCoin.sol#L1376
Status	Acknowledged

Description

The contract admin has the authority to stop the sales for all users. The owner may take advantage of it by calling the pause function. As a result, the contract may operate as a honeypot.

```
Shell
function _update(address from, address to, uint256 amount)
internal virtual override(ERC20Pausable)
```

Recommendation

The team should carefully manage the private keys of the admin accounts. We strongly recommend a powerful security mechanism that will prevent a single user from accessing the contract admin functions.

Temporary Solutions:

These measurements do not decrease the severity of the finding

- Introduce a time-locker mechanism with a reasonable delay.
- Introduce a multi-signature wallet so that many addresses will confirm the action.
- Introduce a governance model where users will vote about the actions.

Permanent Solution:

- Renouncing the authority, which will eliminate the threats but it is non-reversible.

Team Update

The team has acknowledged that this is not a security issue and states:

The admin functions, including the pause functionality, are controlled by a multi-signature wallet requiring 3 out of 5 signatures to execute any transaction. This ensures that no single individual can unilaterally pause the contract or perform any administrative action. The multi-sig setup provides robust protection against both malicious actors and compromised private keys, as any critical operation requires consensus from the majority of key holders. This approach aligns with the audit's recommended temporary solution of introducing a multi-signature wallet for admin functions.

MT - Mints Tokens

Criticality	Minor / Informative
Location	ViFoxCoin.sol#L1516,1677
Status	Acknowledged

Description

The contract allows the minters to mint tokens on registered addresses. Additionally users can stake their tokens to mint more as rewards. The total tokens minted are equal to the `maxSupply`. As a result, the contract tokens will be highly inflated.

Shell

```
function transferVFX(uint256 userId, uint256
vfxPoints, address userWallet) external
onlyRole(MINTER_ROLE) nonReentrant whenNotPaused {
    ...
    _mint(userWallet, totalTokensToMint);
    ...
}
function unstake(uint256 unstakeAmount) external
nonReentrant whenNotPaused {
    ...
    if (mintable > 0) {
        _mint(msg.sender, mintable);
        mintedRewards = mintable;
    }
    ...
}
```

Recommendation

The team should carefully manage the private keys of the minters' accounts. We strongly recommend a powerful security mechanism that will prevent a single user from accessing the contract admin functions.

Temporary Solutions:

These measurements do not decrease the severity of the finding

- Introduce a time-locker mechanism with a reasonable delay.
- Introduce a multi-signature wallet so that many addresses will confirm the action.
- Introduce a governance model where users will vote about the actions.

Permanent Solution:

- Renouncing the ownership, which will eliminate the threats but it is non-reversible.

Team Update

The team has acknowledged that this is not a security issue.

ISU - Incorrect Stake Update

Criticality	Critical
Location	ViFoxCoin.sol#L1650
Status	Acknowledged

Description

The `unstake` function attempts to remove the `userStakes` that have an amount of zero from the `stakes` list. This is performed through a loop. However the loop starts from the last element instead of the first. If the amount is not zero the loop breaks. As a result it is possible that empty stakes are not removed from the array as long as there is at least one `userStake` with non-zero amount after the elements that should be removed.

Shell

```
for (uint256 i = stakes.length; i > 0; i--) {  
    if (stakes[i - 1].amount == 0) {  
        stakes.pop();  
    } else {  
        break;  
    }  
}
```

Recommendation

The team is advised to perform the loop in the same way the stakes are updated, first in , first out.

Team Update

The team has acknowledged that this is not a security issue and states:

The staking mechanism is managed entirely through our official website interface, which holds exclusive access to the MINTER_ROLE. Our backend system handles all stake-related operations with proper validation and state management before interacting with the smart contract. The website ensures correct stake tracking and prevents any edge cases that could arise from the loop order. Users interact with staking only through our platform, where we maintain accurate records and validate all operations before execution.

AUIU - Abstract User ID Usage

Criticality	Medium
Location	ViFoxCoin.sol#L1492
Status	Acknowledged

Description

The `transferVFX` function can be used by minters in order to mint tokens in the `userWallet`. The function accepts the `userId` and `userWallet` as input. However there is no check in the function to ensure that the `userId` points to either the `userWallet` or the minter. As a result the minter can use any `userId` to mint tokens in any address without any restrictions.

```
Shell
function transferVFX(uint256 userId, uint256 vfxPoints,
address userWallet) external onlyRole(MINTER_ROLE)
nonReentrant whenNotPaused {
    ...
    require(registeredUsers[userIdentifiers[userId]],
"User not registered");
    require(userWallet != address(0), "Invalid user wallet
address");
    ...
    userTransfers[userId].push(UserTransfer({
        amount: totalTokensToMint,
        timestamp: block.timestamp,
        to: userWallet
    }));
    ...
    _mint(userWallet, totalTokensToMint);
}
```

Recommendation

The team should ensure that the `transferVFX` function performs validations and checks to ensure that the contract operates as intended.

Team Update

The team has acknowledged that this is not a security issue and states:

The MINTER_ROLE is exclusively granted to our official website's backend infrastructure. The website implements comprehensive validation logic before calling the `transferVFX` function, including verification that the `userId` corresponds to the correct `userWallet` address. Since only our controlled backend can access the minter role, all necessary checks and validations are performed at the application layer before any blockchain transaction is initiated. Unauthorized parties cannot call this function, making exploitation impossible in practice.

PRO - Possible Registration Overwrite

Criticality	Medium
Location	ViFoxCoin.sol#L1480
Status	Acknowledged

Description

The contract allows the minter role to register users through the `registeredUsers` function. This function accepts as input the id of user and the address that will be registered. The function checks if the user is already registered however it does not check if the id is already used. As a result the same id can be used multiple times. In that case the new address will overwrite the previous in the `userIdentifier` mapping and both users will have the same id. Finally the new user will also be registered while not unregistering the previous one.

Shell

```
function registerUser(uint256 userId, address
userIdentifier) external onlyRole(MINTER_ROLE)
whenNotPaused {
    emit RegisterUserAttempt(userId, userIdentifier);
    require(!registeredUsers[userIdentifier], "User
already registered");
    require(userIdentifier != address(0), "Invalid user
identifier address");

    userIdentifiers[userId] = userIdentifier;
    userIds[userIdentifier] = userId;
    registeredUsers[userIdentifier] = true;

    emit UserRegistered(userId, userIdentifier);
}
```

Recommendation

The team should ensure that each `userId` should only be used ones to ensure that the registration process works as intended.

Team Update

The team has acknowledged that this is not a security issue and states:

User registration is performed exclusively through our official website, which has sole access to the MINTER_ROLE. Our backend maintains a database that tracks all registered userIds and prevents any duplicate userId from being submitted to the smart contract. Before calling the registerUser function, our system verifies that the userId has never been used previously. This application-level validation ensures that registration overwrites cannot occur, as duplicate userId submissions are rejected before reaching the blockchain.

TSI - Tokens Sufficiency Insurance

Criticality	Medium
Location	ViFoxCoin.sol#L1668
Status	Acknowledged

Description

During the `unstake` function the contract calculated the `rewardsFromPenalty`. It then transfers this amount to the user. However, the contract does not have a deposit to ensure that this amount is not withdrawn from the stakes of other users. As a result the contract may not have enough tokens to cover the amount or send amounts staked by other users.

```
Shell
uint256 rewardsFromPenalty = 0;
if (penaltyPool > 0) {
    rewardsFromPenalty = penaltyPool >= totalRewards ?
totalRewards : penaltyPool;
    penaltyPool -= rewardsFromPenalty;
    totalRewards -= rewardsFromPenalty;
}
if (rewardsFromPenalty > 0) {
    _transfer(address(this), msg.sender,
rewardsFromPenalty);
}
```

Recommendation

It is recommended to consider implementing a more decentralized approach for handling the contract tokens. One possible solution is to hold a deposit of tokens within the contract itself to ensure that them amount is withdrawn from it instead of other users.

Team Update

The team has acknowledged that this is not a security issue.

CBSU - Calculations Before Supply Update

Criticality	Minor / Informative
Location	ViFoxCoin.sol#L1637
Status	Acknowledged

Description

The `unstake` function calculates the `totalRewards` based on the `rewardPerPeriod`. To find the `rewardPerPeriod` the `calculateRewardPerPeriod` function is called with the remaining supply as input. However this process is performed multiple times before updating the remaining supply. As a result rewards may be calculated based on an outdated `rewardPerPeriod`.

```
Shell
function unstake(uint256 unstakeAmount) external
nonReentrant whenNotPaused {
    ...
    uint256 remainingSupply = getRemainingSupply();
    for (uint256 i = 0; i < stakes.length &&
remainingUnstakeAmount > 0; i++) {
        if (stakingDuration < 10 days) {
            ...
        } else {
            uint256 fullPeriods = stakingDuration / 10
days;
            uint256 rewardPerPeriod =
calculateRewardPerPeriod(lotPortion, remainingSupply);
            totalRewards += rewardPerPeriod * fullPeriods;
        }
    }
}
```

Recommendation

The team is advised to use a local variable that updates after every loop to ensure that rewards are calculated properly.

Team Update

The team has acknowledged that this is not a security issue.

CO - Code Optimization

Criticality	Minor / Informative
Location	ViFoxCoin.sol#L1410,1615
Status	Acknowledged

Description

There are code segments that could be optimized. A segment may be optimized so that it becomes a smaller size, consumes less memory, executes more rapidly, or performs fewer operations.

Shell

```
function canMintTokens(uint256 vfxPoints) public view  
returns (bool canMint, uint256 totalTokensToMint, bool  
exceedsSupply, bool exceedsDailyLimit, string memory  
reason)  
function unstake(uint256 unstakeAmount) external  
nonReentrant whenNotPaused
```

Recommendation

The team is advised to take these segments into consideration and rewrite them so the runtime will be more performant. That way it will improve the efficiency and performance of the source code and reduce the cost of executing it.

Team Update

The team has acknowledged that this is not a security issue.

CCR - Contract Centralization Risk

Criticality	Minor / Informative
Location	ViFoxCoin.sol
Status	Acknowledged

Description

The contract's functionality and behavior are heavily dependent on external parameters or configurations. While external configuration can offer flexibility, it also poses several centralization risks that warrant attention. Centralization risks arising from the dependence on external configuration include Single Point of Control, Vulnerability to Attacks, Operational Delays, Trust Dependencies, and Decentralization Erosion.

Recommendation

To address this finding and mitigate centralization risks, it is recommended to evaluate the feasibility of migrating critical configurations and functionality into the contract's codebase itself. This approach would reduce external dependencies and enhance the contract's self-sufficiency. It is essential to carefully weigh the trade-offs between external configuration flexibility and the risks associated with centralization.

Team Update

The team has acknowledged that this is not a security issue.

HV - Hardcoded Values

Criticality	Minor / Informative
Location	ViFoxCoin.sol#L1425,1701
Status	Acknowledged

Description

The contract contains multiple instances where numeric values are directly hardcoded into the code logic rather than being assigned to constant variables with descriptive names. Hardcoding such values can lead to several issues, including reduced code readability, increased risk of errors during updates or maintenance, and difficulty in consistently managing values throughout the contract. Hardcoded values can obscure the intent behind the numbers, making it challenging for developers to modify or for users to understand the contract effectively.

```
Shell
uint256[5] memory thresholds = [
    11_000_000 * 10 ** DECIMALS,
    9_000_000 * 10 ** DECIMALS,
    7_000_000 * 10 ** DECIMALS,
    5_000_000 * 10 ** DECIMALS,
    3_000_000 * 10 ** DECIMALS
];
uint256[5] memory ratios = [
    uint256(5),
    uint256(10),
    uint256(20),
    uint256(50),
    uint256(100)
];
...
if (remainingSupply >= 11_000_000 * unit) return amount /
370;
if (remainingSupply >= 9_000_000 * unit) return amount /
526;
if (remainingSupply >= 7_000_000 * unit) return amount /
```

```
740;  
if (remainingSupply >= 5_000_000 * unit) return amount /  
1250;  
if (remainingSupply >= 3_000_000 * unit) return amount /  
1800;  
return amount / 2500;
```

Recommendation

It is recommended to replace hardcoded numeric values with variables that have meaningful names. This practice improves code readability and maintainability by clearly indicating the purpose of each value, reducing the likelihood of errors during future modifications. Additionally, consider using constant variables which provide a reliable way to centralize and manage values, improving gas optimization throughout the contract.

Team Update

The team has acknowledged that this is not a security issue.

MEE - Missing Events Emission

Criticality	Minor / Informative
Location	ViFoxCoin.sol#L1520,1545
Status	Acknowledged

Description

The contract performs actions and state mutations from external methods that do not result in the emission of events. Emitting events for significant actions is important as it allows external parties, such as wallets or dApps, to track and monitor the activity on the contract. Without these events, it may be difficult for external parties to accurately determine the current state of the contract.

Shell

```
function setDailyMintLimit(uint256 newLimit) external  
onlyRole(ADMIN_ROLE)  
function resetDailyMint() external onlyRole(ADMIN_ROLE)  
  
}
```

Recommendation

It is recommended to include events in the code that are triggered each time a significant action is taking place within the contract. These events should include relevant details such as the user's address and the nature of the action taken. By doing so, the contract will be more transparent and easily auditable by external parties. It will also help prevent potential issues or disputes that may arise in the future.

Team Update

The team has acknowledged that this is not a security issue.

PAMAR - Pair Address Max Amount Restriction

Criticality	Minor / Informative
Location	ViFoxCoin.sol#L1380
Status	Acknowledged

Description

The contract is configured to enforce a maximum token accumulation limit through checks. This mechanism aims to prevent excessive token concentration by reverting transactions that overcome the specified cap. However, this functionality encounters issues when transactions default to the pair address during sales. If the pair address is not listed in the exceptions, then the sale transactions are inadvertently stopped, effectively disrupting operations and making the contract susceptible to unintended behaviors akin to a honeypot.

Shell

```
if (vest.totalAllocation > 0) {  
    uint256 allowed = transferableAmount(from);  
    require(amount <= allowed, "Vesting: amount exceeds  
vested tokens");  
    vest.released += amount;  
}
```

Recommendation

It is advised to modify the contract to ensure uninterrupted operations by either permitting the pair address to exceed the established token accumulation limit or by safeguarding its status in the exception list. By recognizing and allowing these essential addresses the flexibility to hold more tokens than typical limits, the contract can maintain seamless transaction flows and uphold the liquidity and stability of the ecosystem. This modification is vital for avoiding disruptions that could impact the functionality and security of the contract.

Team Update

The team has acknowledged that this is not a security issue.

PSU - Potential Subtraction Underflow

Criticality	Minor / Informative
Location	ViFoxCoin.sol#L326
Status	Acknowledged

Description

The contract subtracts two values, the second value may be greater than the first value if the contract owner misuses the configuration. As a result, the subtraction may underflow and cause the execution to revert.

Specifically there is no check to ensure that `totalAllocated` are equal or less than the `initialSupply`.

```
Shell
uint256 deployerSupply = initialSupply - totalAllocated;
```

Recommendation

The team is advised to properly handle the code to avoid underflow subtractions and ensure the reliability and safety of the contract. The contract should ensure that the first value is always greater than the second value. It should add a sanity check in the setters of the variable or not allow executing the corresponding section if the condition is violated.

Team Update

The team has acknowledged that this is not a security issue.

RSK - Redundant Storage Keyword

Criticality	Minor / Informative
Location	ViFoxCoin.sol#L1710
Status	Acknowledged

Description

The contract uses the `storage` keyword in a view function. The `storage` keyword is used to persist data on the contract's storage. View functions are functions that do not modify the state of the contract and do not perform any actions that cost gas (such as sending a transaction). As a result, the use of the `storage` keyword in view functions is redundant.

```
Shell
function getTotalStaked(address account) public view
returns (uint256) {
    ...
    Stake[] storage stakes = userStakes[account];
    ...
}

    mintedRewards = mintable;
}
...
}
```

Recommendation

It is generally considered good practice to avoid using the `storage` keyword in view functions because it is unnecessary and can make the code less readable.

Team Update

The team has acknowledged that this is not a security issue.

RVF - Redundant View Functionality

Criticality	Minor / Informative
Location	ViFoxCoin.sol#L1562
Status	Acknowledged

Description

The contract contains multiple functions that return public state variables. If a variable is declared public a getter function is automatically generated during deployment that allows users to retrieve their value. As a result the following functions are redundant.

Shell

```
function getAddressByUserId(uint256 userId) external  
view returns (address)  
function getUserIdByWalletAddress(address walletAddress)  
external view returns (uint256)  
function getDailyMintLimit() external view returns  
(uint256)  
function getDailyMinted() external view returns (uint256)  
function isPaused() external view returns (bool)  
function getLastMintReset() external view returns  
(uint256)
```

Recommendation

The team is advised to refactor the contract and remove view functions that are already generated through the public state.

Team Update

The team has acknowledged that this is not a security issue.

UPS - Unsanitized Parameter Setters

Criticality	Minor / Informative
Location	ViFoxCoin.sol#L1521
Status	Acknowledged

Description

The contract contains functionality that allows the admin address to change the contract's configurations without proper validation in the inputs.

Shell

```
function setDailyMintLimit(uint256 newLimit) external  
onlyRole(ADMIN_ROLE)  
function defineOperator(address operator) external  
onlyRole(ADMIN_ROLE)  
function changeOperator(address oldOperator, address  
newOperator) external onlyRole(ADMIN_ROLE)  
function removeOperator(address operator) external  
onlyRole(ADMIN_ROLE)  
function transferOwnership(address newOwner) external  
onlyRole(ADMIN_ROLE)
```

Recommendation

The team is advised to enhance these functions with restrictions that the contract always has the correct configurations.

Team Update

The team has acknowledged that this is not a security issue.

L02 - State Variables could be Declared Constant

Criticality	Minor / Informative
Location	ViFoxCoin.sol#L1234
Status	Acknowledged

Description

State variables can be declared as constant using the constant keyword. This means that the value of the state variable cannot be changed after it has been set. Additionally, the constant variables decrease gas consumption of the corresponding transaction.

Shell

```
6 public maxSupply = 21000000 * 10 ** DECIMALS;
```

Recommendation

Constant state variables can be useful when the contract wants to ensure that the value of a state variable cannot be changed by any function in the contract. This can be useful for storing values that are important to the contract's behavior, such as the contract's address or the maximum number of times a certain function can be called. The team is advised to add the constant keyword to state variables that never change.

Team Update

The team has acknowledged that this is not a security issue.

L09 - Dead Code Elimination

Criticality	Minor / Informative
Location	ViFoxCoin.sol#L143,547,1069
Status	Acknowledged

Description

In Solidity, dead code is code that is written in the contract, but is never executed or reached during normal contract execution. Dead code can occur for a variety of reasons, such as:

- Conditional statements that are always false.
- Functions that are never called.
- Unreachable code (e.g., code that follows a return statement).

Dead code can make a contract more difficult to understand and maintain, and can also increase the size of the contract and the cost of deploying and interacting with it.

```
Shell
function _contextSuffixLength() internal view virtual
returns (uint256) {
    return 0;
}

on _burn(address account, uint256 value) internal {
    if (account == address(0)) {
        revert ERC20InvalidSender(address(0));
    }
    _update(account, address(0), value);
}
```

Recommendation

To avoid creating dead code, it's important to carefully consider the logic and flow of the contract and to remove any code that is not needed or that is never executed. This can help improve the clarity and efficiency of the contract.

Team Update

The team has acknowledged that this is not a security issue.

L13 - Divide before Multiply Operation

Criticality	Minor / Informative
Location	ViFoxCoin.sol#L1636,1638
Status	Acknowledged

Description

It is important to be aware of the order of operations when performing arithmetic calculations. This is especially important when working with large numbers, as the order of operations can affect the final result of the calculation. Performing divisions before multiplications may cause loss of precision.

Shell

```
6 fullPeriods = stakingDuration / 10 days;  
  
rewards += rewardPerPeriod * fullPeriods;
```

Recommendation

To avoid this issue, it is recommended to carefully consider the order of operations when performing arithmetic calculations in Solidity. It's generally a good idea to use parentheses to specify the order of operations. The basic rule is that the multiplications should be prior to the divisions.

Team Update

The team has acknowledged that this is not a security issue.

L18 - Multiple Pragma Directives

Criticality	Minor / Informative
Location	ViFoxCoin.sol#L12,94,122,152,317,630,731,759,788,999,1079,1193,1226
Status	Acknowledged

Description

If the contract includes multiple conflicting pragma directives, it may produce unexpected errors. To avoid this, it's important to include the correct pragma directive at the top of the contract and to ensure that it is the only pragma directive included in the contract.

```
Shell
pragma solidity ^0.8.20;
    solidity ^0.8.20;
...
/*
    solidity ^0.8.0;

**
```

Recommendation

It is important to include only one pragma directive at the top of the contract and to ensure that it accurately reflects the version of Solidity that the contract is written in.

By including all required compiler options and flags in a single pragma directive, the potential conflicts could be avoided and ensure that the contract can be compiled correctly.

Team Update

The team has acknowledged that this is not a security issue.

L19 - Stable Compiler Version

Criticality	Minor / Informative
Location	ViFoxCoin.sol#L94,999
Status	Acknowledged

Description

The `^` symbol indicates that any version of Solidity that is compatible with the specified version (i.e., any version that is a higher minor or patch version) can be used to compile the contract. The version lock is a mechanism that allows the author to specify a minimum version of the Solidity compiler that must be used to compile the contract code. This is useful because it ensures that the contract will be compiled using a version of the compiler that is known to be compatible with the code.

```
Shell
pragma solidity ^0.8.20;

solidity ^0.8.0;

/**
```

Recommendation

The team is advised to lock the pragma to ensure the stability of the codebase. The locked pragma version ensures that the contract will not be deployed with an unexpected version. An unexpected version may produce vulnerabilities and undiscovered bugs. The compiler should be configured to the lowest version that provides all the required functionality for the codebase. As a result, the project will be compiled in a well-tested LTS (Long Term Support) environment.

Team Update

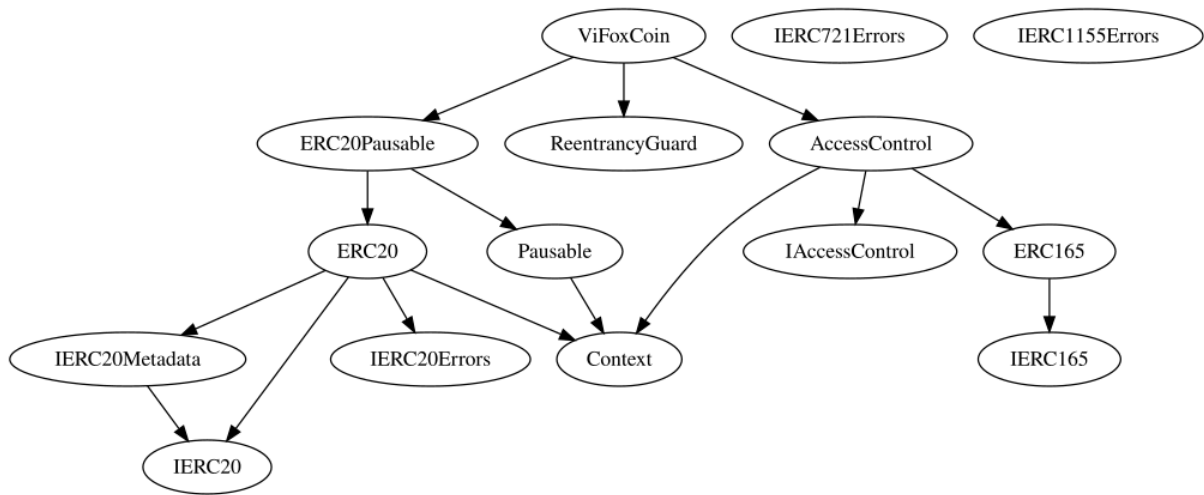
The team has acknowledged that this is not a security issue.

Functions Analysis

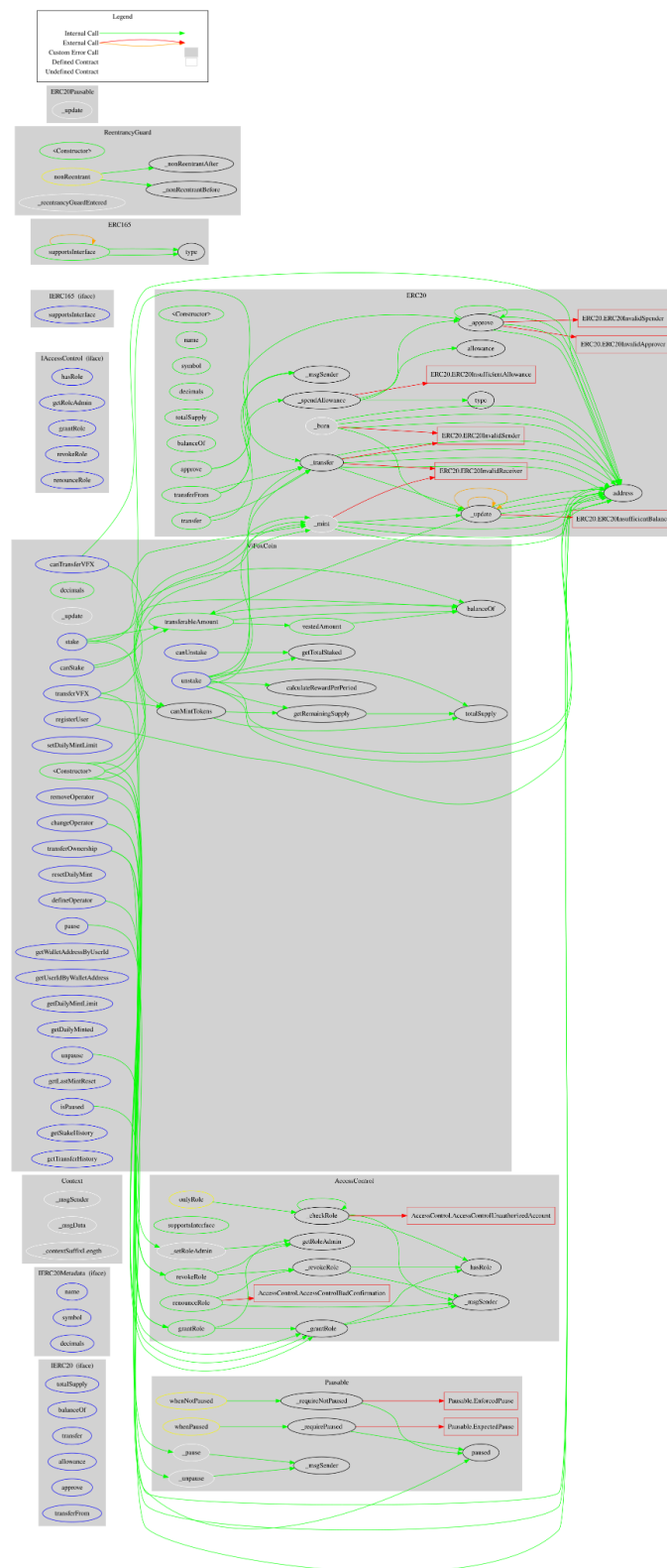
Contract	Type	Bases		
	Function Name	Visibility	Mutability	Modifiers
ViFoxCoin	Implementation	ERC20Pausable, AccessControl, ReentrancyGuard		
		Public	✓	ERC20
	decimals	Public		-
	vestedAmount	Public		-
	transferableAmount	Public		-
	_update	Internal	✓	
	canTransferVFX	External		-
	canMintTokens	Public		-
	canStake	External		-
	canUnstake	External		-
	registerUser	External	✓	onlyRole whenNotPaused
	transferVFX	External	✓	onlyRole nonReentrant whenNotPaused
	setDailyMintLimit	External	✓	onlyRole
	defineOperator	External	✓	onlyRole
	changeOperator	External	✓	onlyRole
	removeOperator	External	✓	onlyRole

	transferOwnership	External	✓	onlyRole
	resetDailyMint	External	✓	onlyRole
	pause	External	✓	onlyRole
	unpause	External	✓	onlyRole
	getRemainingSupply	Public		-
	getWalletAddressByUserId	External		-
	getUserIdByWalletAddress	External		-
	getDailyMintLimit	External		-
	getDailyMinted	External		-
	isPaused	External		-
	getLastMintReset	External		-
	stake	External	✓	nonReentrant whenNotPaused
	unstake	External	✓	nonReentrant whenNotPaused
	calculateRewardPerPeriod	Internal		
	getTotalStaked	Public		-
	getStakeHistory	External		-
	getTransferHistory	External		-

Inheritance Graph



Flow Graph



Summary

ViFoxCoin contract implements a token, staking and locker mechanism. This audit investigates security issues, business logic concerns and potential improvements. There are some functions that can be abused by the admin like stop transactions and mint tokens. if the contract owner abuses the mint functionality, then the contract will be highly inflated. A multi-wallet signing pattern will provide security against potential hacks. Temporarily locking the contract or renouncing ownership will eliminate all the contract threats. The team has acknowledged all findings.

Disclaimer

The information provided in this report does not constitute investment, financial or trading advice and you should not treat any of the document's content as such. This report may not be transmitted, disclosed, referred to or relied upon by any person for any purposes nor may copies be delivered to any other person other than the Company without Cyberscope's prior written consent. This report is not nor should be considered an "endorsement" or "disapproval" of any particular project or team. This report is not nor should be regarded as an indication of the economics or value of any "product" or "asset" created by any team or project that contracts Cyberscope to perform a security assessment. This document does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors' business, business model or legal compliance. This report should not be used in any way to make decisions around investment or involvement with any particular project. This report represents an extensive assessment process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. Cyberscope's position is that each company and individual are responsible for their own due diligence and continuous security. Cyberscope's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies and in no way claims any guarantee of security or functionality of the technology we agree to analyze. The assessment services provided by Cyberscope are subject to dependencies and are under continuing development. You agree that your access and/or use including but not limited to any services reports and materials will be at your sole risk on an as-is where-is and as-available basis. Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives and other unpredictable results. The services may access and depend upon multiple layers of third parties.

About Cyberscope

Cyberscope is a TAC blockchain cybersecurity company that was founded with the vision to make web3.0 a safer place for investors and developers. Since its launch, it has worked with thousands of projects and is estimated to have secured tens of millions of investors' funds.

Cyberscope is one of the leading smart contract audit firms in the crypto space and has built a high-profile network of clients and partners.



A **TAC Security** Company

The Cyberscope team

cyberscope.io